1. (2 pontos) Assinale **V** (verdadeiro) ou **F** (falso) para as seguintes alternativas verdadeira: (responda aqui mesmo, não é necessário provar)

( ) *Se f(n) ∈ θ(g(n)) então g(n) ∈ θ(f(n))*
( ) *Se f(n) ∈ O(g(n)) então g(n) ∈ ω(f(n))*
( ) $2^n \in O(n)$
( ) $2^n \in O(n^n)$
( ) $\mathrm{n}^2 - \mathrm{n} \in \omega(n)$
( ) $\mathrm{n} \in \omega(n)$
( ) $\mathrm{n} \in \omega(\log(n))$
( ) $\log(\mathrm{n}) + \mathrm{n}^2 \in \Omega(\log(n))$
( ) $\mathrm{n}*\log(\mathrm{n}) \in \mathrm{O}((\log(n))^2)$
( ) $\log_{10}\mathrm{n} \in \theta(\log_2\mathrm{n})$

2.

a) (2,5 pontos) Escreva uma função recursiva `minMax`, utilizando o paradigma de divisão e conquista que calcula o elemento mínimo e o elemento máximo de um vetor com n números inteiros. Sugestão: as variáveis que receberão a resposta podem ser parâmetros dessa função, tendo ela o seguinte protótipo:

```
void minMax(int* min, int* max);
```

b) (0,5 ponto) Apresente a equação de recorrência que descreve a complexidade do seu algoritmo.

3. Você fez um acordo com os seus pais, no 1º mês você recebe R$10,00 de mesada, no 2º R$15,00 e no 3º R$18,00. A partir do 4º mês você recebe a soma das mesadas dos 3 meses anteriores.

a) (0,5 ponto) Escreva a equação de recorrência que define a mesada(n), que é a função de quanto receberá de mesada no mês n, e identifique os casos base.

b) (2,0 pontos) Escreva em C ou em pseudocódigo um algoritmo para calcular a mesada(n) com programação dinâmica.

(CONTINUA NO VERSO)

4. (2,5 pontos) Um determinado algoritmo possui o tempo de execução descrito pela recorrência

$$T(n) = 4\,T(n/2) + n^3$$

sendo T(1) = O(1). Qual a complexidade desse algoritmo? Calcule utilizando o método de sua escolha: expansão da recorrência, substituição ou o Teorema Mestre (enunciado a seguir).

Teorema Mestre

Sejam a ≥ 1 e b > 1 constantes, seja f(n) uma função e seja T(n) definida sobre os inteiros não negativos pela recorrência

T(n) = aT(n/b) +f(n)

onde interpretamos n/b com o significado de $\lfloor n/b \rfloor$ *ou* $\lceil n/b \rceil$

Então, T(n) pode ser limitado assintoticamente como a seguir:

1. Se $f(n) = O(n^{\log_b a - \varepsilon})$ para alguma constante ε > 0, então $T(n) = \Theta(n^{\log_b a})$
2. Se $f(n) = \Theta(n^{\log_b a})$, então $T(n) = \Theta(n^{\log_b a} \lg n)$
3. Se $f(n) = \Omega(n^{\log_b a + \varepsilon})$ para alguma constante ε > 0,
   e se **af(n/b) ≤ cf(n)** para alguma constante **c < 1** e para todo ***n*** suficientemente grande, então $T(n) = \Theta(f(n))$

BOA PROVA!!!